

The ReACT Agent Loop (Per-Turn Reason + Act + Observe)

Scope. This document describes the **core ReACT agent loop** — the per-turn loop that runs every time you send a message to ragent. It is *not* the goal-driven /loop feature (see TUI-QUICKSTART.md, section 5, for /loop). Where the two interact, the interaction is called out explicitly.

1. What it is

Every user message triggers an agentic **Reason -> Act -> Observe** cycle:

1. **Reason** — the session processor streams an LLM response for the current conversation state.
2. **Act** — if the model requests tool calls, each call is permission-checked and executed.
3. **Observe** — the tool results are appended to the conversation history and fed back into the next LLM request.

The cycle repeats until the model produces an answer **without** any tool calls, or a termination condition fires (step limit, cancel, interrupt, or an unrecoverable error).

Element	Location
Orchestrator struct	SessionProcessor — crates/ragent-agent/src/session/processor.rs:106
Public entry (plain text prompt)	process_message — processor.rs:1137
Public entry (pre-built Message, multipart/images)	process_user_message — processor.rs:1164
The core loop	inline loop { ... } — processor.rs:1472, inside process_user_message
Per-step helpers	crates/ragent-agent/src/session/loop_steps.rs — prepare_client (:121), build_turn_system_prompt (:362), build_turn_chat_messages (:649), run_inline_init_acknowledgement (:739), call_llm_step (:857), finalize_assistant_message (:1529)
Loop-run state machine	LoopSpec / LoopTracker / StopCondition — crates/ragent-agent/src/session/loop_state.rs

SessionProcessor orchestrates the loop: it accepts a user message, streams an LLM response, executes any requested tool calls, and iterates until the model signals completion or the step limit is reached.

2. Turn lifecycle

Phase A — turn setup (before the loop)

Location: processor.rs:1171-1465.

1. **Telemetry** — an `agent_loop_profiler()` and a `SessionRecorder` are created and `record_session_start()` is logged.
2. **User message persisted** — `storage_op(move |s| s.create_message(&msg))` writes the message to SQLite on a blocking thread; `Event::MessageStart` is published.
3. **RunId issued** — `RunId::new()`; the message is recorded in the activity log as a user model message. This `run_id` links every event of the turn.
4. **Client/model/config resolution** — `prepare_client` resolves the provider client, model reference, session config, working directory and team context. Failure here is an unrecoverable provider-stage error: the run is terminated with status error and `Err` is returned.
5. **Per-run cost tracking** — a `TokenUsage` listener accumulates (`input`, `output`) tokens for the turn; when the turn ends a `RunCostSummary` row is computed from merged price entries and published as `Event::RunCostSummary`.
6. **System prompt build** — `build_turn_system_prompt` assembles the agent preset, initiatives, tool reference, codeindex guidance and team guidance via the cached `SystemPromptCache`.
7. **Chat history build** — `build_turn_chat_messages` returns the provider chat messages, whether compression happened, the last reported input token count, and the context window. The vector is shared behind an `Arc<Vec<ChatMessage>>` so per-retry requests share it by refcount.
8. **AGENTS.md init acknowledgement** — `run_inline_init_acknowledgement` prints a one-time display-only acknowledgement (skipped for sub-agents).
9. **Loop setup:**
 - `max_steps = agent.max_steps.unwrap_or(1024)` — the default iteration ceiling is **1024** unless the agent preset sets one.
 - `event_bus.set_step(session_id, 0)`.
 - Tool definitions are chosen: **empty if `max_steps <= 1`** (single-shot: the model gets no tools); filtered to the active `/loop` tool set when one is active; otherwise the full cached tool registry.
 - A reusable assistant placeholder `Message` is created and persisted — its id anchors all streaming updates to one DB row.
 - Interrupt plumbing: `interrupt_requested = user cancel flag OR active /loop interrupt flag`.

Phase B — one ReACT iteration

Each pass of the loop { body (`processor.rs:1472+`) runs this guard chain in order:

#	Guard	Behaviour
1	Stop-flag guard	If the loop tracker <code>is_stopped()</code> , break — no stage runs.
2	Safe-point interrupt	On <code>/loop</code> runs, a user cancel or a raised <code>/loop</code> interrupt breaks before the budget gate (first stop wins; user intervention takes precedence) and ends the run with status interrupted.

#	Guard	Behaviour
3	Budget gate	<code>tracker.begin_step()</code> returns false when the <code>/loop</code> step or token budget was reached; the run ends as <code>budget_exhausted</code> before another LLM request is sent .
4	Step counter	<code>set_step(current + 1)</code> under the <code>loop.step.total</code> profile scope.
5	Max-steps check	<code>step > max_steps -> warning</code> + <code>Event::AgentError("Reached maximum steps (N)") +</code> <code>break</code> .
6	User-cancel check	On plain (non-loop) turns, a set cancel flag logs the timing breakdown, persists the partial assistant message, publishes <code>Event::MessageEnd { reason: Cancelled }</code> , records <code>TerminationReason::Interrupted</code> , publishes the run-cost summary, and returns an empty message. On <code>/loop</code> runs this branch is effectively plain-turn-only — a loop-run cancel already ended the run at guard 2.

Then the reasoning stage:

7. **Event::ToolsSent** — published **only on step 1 and only when tools are offered** (`!tool_definitions.is_empty()`), so the TUI renders the offered tool list once.
8. **Pre-send auto-compaction** — when `compaction.auto` is enabled and not yet attempted this turn, the request is token-estimated and `evaluate_trigger` runs against the context window; if the trigger fires, history is summarised (`crate::compaction::compact`), a synthetic `Role::Compaction` message is persisted, and the in-memory history is swapped to compaction-summary + verbatim recent tail. Compaction failure only warns and continues uncompressed.
9. **LLM call** — `call_llm_step` builds the `ChatRequest` (system prompt, history, tool definitions) and folds the provider stream (`futures::StreamExt`) into incremental `text_buffer`, `reasoning_buffer` and accumulated `tool_calls`. A fatal stream error terminates the run with status error.
10. **Provider-reported input tokens** — persisted into session state so the TUI context panel shows the same figure next turn.
11. **Token tally** — `tracker.record_tokens(...)` is persisted; the *next* iteration's budget gate sees the updated totals.
12. **Response parts** — non-empty reasoning becomes a `MessagePart::Reasoning` on the assistant

message; non-empty text publishes `Event::ModelResponse` (preview, elapsed ms, token counts) and becomes a `MessagePart::Text`.

Phase C — decision point: tool call vs final answer

```
if llm_result.tool_calls.is_empty()
  -> the text response IS the final answer; the loop ends
else
  -> tool dispatch phase; afterwards `continue` back to Phase B
```

When the model answers **without** tool calls, three sub-paths run (in order):

- **Sub-agent summary nudge** — if this is a sub-agent run past step 1 whose text is under the 2000-byte narration limit, the narration was likely “Now let me check...” rather than findings. The narration is popped, pushed back into history as an assistant message, and a `SUBAGENT_SUMMARY_NUDGE` user message requests the real summary; the loop continues. This happens **once per run** and is *not* a termination.
- **Verification gate** (/loop only, T-005) — when a spec with a `verify_cmd` is active, `run_verification_command` decides: **pass** -> the run ends as `GoalAchieved` with the `verify` label; **fail with steps remaining** -> the failure output becomes the next observation and the loop continues (a success record resets the consecutive-failure counter so unrelated recoverable failures do not burn the retry allowance); **fail with budget breached** -> ends as `budget_exhausted`.
- **Plain stop** — a no-tool-call response with no verification command means the goal was reached -> run ends as `GoalAchieved`.

When tool calls are **non-empty**, one more interrupt safe point runs, then the **tool dispatch phase** (`processor.rs:2042+`): text is moved (not cloned) into a `ContentPart::Text`, `parallel_tool_calls` is read from `experimental.parallel_tool_calls`, one `ToolContext` is built per step, and per tool call: **permission check** -> **execution** -> **result collection** -> **interim save**, then continue back to the top of the loop so the next LLM request carries the observations.

3. Flow diagram

```
User message
|
v
persist message -> RunId issued -> prepare_client (fatal on error)
|
v
build system prompt + chat history (cached)
|
+-----+
| loop { (processor.rs:1472)                                |
|   stop-flag? -> break                                     |
|   /loop interrupt (or cancel on loop runs)?              |
|   -> status interrupted, break                             |
|   budget breach? -> budget_exhausted, break              |
|   step > max_steps? -> AgentError, break                  |
| }                                                         |
+-----+
```

```

|   plain-turn cancel? -> persist partial,           |
|       MessageEnd{Cancelled}                         |
|   ToolsSent (step 1 only)                           |
|   auto-compaction (if enabled, once per turn)       |
|   call_llm_step <-- REASON (stream fold)           |
|   record tokens / response parts                    |
|                                                     |
|   tool_calls empty? --yes--> final answer, end      |
|       |no                                           |
|       v                                           |
|   interrupt safe point                             |
|   per tool: permission -> execute -> observe        |
|   append tool results -> continue loop             |
+-----+

```

4. Tool calls vs final answer

The decision is purely structural: a streamed response that accumulates one or more `tool_calls` is an **action** turn; a response with **no** tool calls is the **final answer** and terminates the loop. Thinking/reasoning blocks are buffered separately (`reasoning_buffer`) and persisted as `MessagePart::Reasoning` — they never terminate the loop on their own.

Tool-call arguments arrive incrementally; assembly is managed through `PendingToolCall` (`history.rs`) before a call is promoted into `llm_result.tool_calls` and dispatched.

5. Permissions and tool execution

Tool execution is gated by the multi-layered permission system:

- `PermissionChecker` consults global rules (permission array in `ragent.json`) and per-agent rules (`agent.<name>.permission`).
- Allow/Deny verdicts are honoured; an Allow that would auto-approve can still be forced into an interactive prompt when a `/loop spec with checkpoints: true` is active (destructive tools are checkpointed).
- `--yes` / `--no-prompt` (CLI) or `/yolo` (TUI) auto-approve every prompt; explicit Deny rules still take precedence over allow rules.
- A permission prompt times out to **deny** (safe default).

A tool task that neither finishes nor panics is killed at the tool watchdog timeout (`TOOL_WATCHDOG_TIMEOUT` = 2000 s) and the whole run is terminated with status `error`; an ordinary task join failure is logged and skipped instead.

6. Streaming, stalls, retries

- **Stream driver:** `call_llm_step` folds `StreamEvents` into the per-step working state (`LoopState`): incremental text, reasoning, tool calls.

- **Thinking blocks** are configured per provider/model (see Section 8, thinking) and persisted as `MessagePart::Reasoning`.
- **Stall detection** — `stream.timeout_secs` bounds the gap between subsequent stream deltas; a stall triggers the retry path. Stalls are detected via `stall_pattern_set()` with `should_flush`.
- **Retry/backoff** — `stream.max_retries` attempts with linear backoff ($N * \text{stream.retry_backoff_secs}$). Retryability helpers: `should_retry_stream_error`, `stream_has_meaningful_partial_output`, `is_token_overflow_error_message`, `is_permanent_llm_api_error`.
- **Initial response timeout** — `stream.initial_response_timeout_secs` waits for the first byte (RTT + provider cold start).

UI event fan-out during a turn: `MessageStart` -> step counter -> `ToolsSent` (step 1) -> `ModelResponse` -> per-tool `ToolCallArgs` / `ToolCallEnd` / `ToolCallStart` (inside the spawned tool task; `ToolCallBatch` at step end) -> `MessageEnd`.

7. Termination conditions

#	Condition	Signal / status
1	Model answers without tool calls	<code>GoalAchieved</code> (plain turns simply return the message)
2	/loop verification gate passes	<code>GoalAchieved</code> with the verify label
3	Step budget exhausted	<code>AgentError</code> "Reached maximum steps (N)"; default ceiling 1024 (agent override) / <code>--maxsteps</code> cap
4	/loop token cost budget exhausted	<code>BudgetExhausted</code> , fired before the next request
5	User cancel	<code>FinishReason::Cancelled</code> , partial message persisted
6	Human interrupt (Esc during a /loop run)	<code>HumanIntervention</code> -> status interrupted; takes precedence over budget at safe points
7	Unrecoverable provider error (prep or stream stage)	status error, <code>Err</code> returned
8	Tool watchdog	tool stalls past 2000 s -> run terminated

Not terminations: the one-shot sub-agent summary nudge, and verification failures that still have steps remaining (they become the next observation).

8. Configuration options

ragent reads configuration from `ragent.json` (or `ragent.jsonc`) with the following precedence, each later layer merged over the previous:

1. Compiled defaults
2. Global file: `~/.config/ragent/ragent.json`
3. Project file: `./.ragent/ragent.json` (created with defaults if neither file exists)
4. `RAGENT_CONFIG` env var -> file path overlay
5. `RAGENT_CONFIG_CONTENT` env var -> inline JSON overlay

Per-provider blocks and per-model entries are deep-merged, so partial overrides preserve lower layers. Resolved config is cached keyed on the mtime + size of every contributing file plus the cwd (`CachedConfigFile`); every `Config::save()` invalidates that cache so same-process writes are immediately visible to subsequent loads.

Core loop knobs

Key	Type	Default	Behaviour
<code>defaultAgent</code>	String	"general"	Default agent preset driving the loop.
<code>agent.<name>.max_steps</code>	<code>Option<u32></code>	unset (falls back to 1024)	Iteration ceiling for the loop when this agent is active.
<code>agent.<name>.model</code>	String	unset	<code>provider:model</code> override for this agent.
<code>agent.<name>.temperature</code>	<code>f32</code>	unset	Sampling temperature.
<code>agent.<name>.top_p</code>	<code>f32</code>	unset	Nucleus sampling.
<code>agent.<name>.permissions</code>	rules	[]	Per-agent permission rules.
<code>permission</code>	<code>Vec<PermissionRule></code>	[]	Global allow/deny/ask rules (permission, pattern, action).
<code>yolo</code>	bool	false	Trust mode: auto-approve everything.
<code>experimental.parallel_tool_calls</code>	bool	false	Execute requested tool calls in parallel rather than sequentially.
<code>experimental.max_background_agents</code>	usize	8	Concurrent background sub-agent cap.
<code>experimental.background_agent_timeout</code>	u64	3600	Sub-agent timeout (seconds).

agent_perf — per-step performance tuning

Consulted at loop startup and on every step. `validate()` refuses to start on violations.

Key	Type	Default	Behaviour
<code>agent_perf.enabled</code>	bool	true	Master switch; false short-circuits every perf optimisation.

Key	Type	Default	Behaviour
agent_perf.profiling	bool	false	Per-scope timing logs at info level.
agent_perf.step_budget	u64secs	300	Max wall-clock seconds per agent step (≥ 5).
agent_perf.stall_timeout	u64secs	60	Seconds without a stream delta before stall recovery (≥ 5).
agent_perf.max_concurrent_tools	u32	min(available_parallelism, 4)	Max parallel tool calls per turn (≥ 1).
agent_perf.parallel_independent_tools	bool	true	Execute independent tool calls in parallel.

stream — request timeouts and retries

Key	Type	Default	Behaviour
stream.initial_response_timeout_secs	u64	300	Wait for the first byte .
stream.timeout_secs	u64	120	Max gap between subsequent deltas before stall -> retry.
stream.max_retries	u32	4	Retry attempts after stall/connection failure (≤ 32).
stream.retry_backoff_secs	u64	2	Attempt N waits N * <code>retry_backoff_secs</code> .

Validation: both timeouts ≥ 5 ; `initial_response_timeout_secs` $\geq$ `timeout_secs`.

compaction — history compaction

Key	Type	Default	Behaviour
compaction.auto	bool	true	Auto-compact before a send when the trigger fires (once per turn).
compaction.threshold	f32	0.7	Fraction of context window that triggers compaction.
compaction.buffer	f32	0.10	Headroom kept after compaction.
compaction.keep.tokens	f32	0.20	Fraction of recent tokens kept verbatim after compaction.

thinking — reasoning blocks

Configured per provider and per model; model-level overrides provider-level.

Key	Type	Default	Behaviour
provider.<id>.thinking.enabled	boolean	provider-dependent	Enable thinking blocks for the provider.
provider.<id>.thinking.effort	String	provider-dependent	Reasoning effort (e.g. low/medium/high).
provider.<id>.models.<id>.thinking.budget.tokens	int	provider-dependent	Token budget for reasoning.

Tool visibility

tool_visibility switches hide tool families from tool definitions and prompt listings (the tools stay registered and executable):

Key	Default
tool_visibility.office	false
tool_visibility.github	false
tool_visibility.gitlab	false
tool_visibility.teams	false
tool_visibility.agents	false
tool_visibility.plan	false
tool_visibility.codeindex	true
tool_visibility.masterfetch	true
tool_visibility.browser	true
tool_visibility.finance	true

The loop section — /loop only, not the core loop

The loop section (loop.max_steps, loop.cost_limit, loop.error_retry_allowance, loop.checkpoints, loop.checkpoint_timeout_secs) configures the **goal-driven /loop feature**, which injects stop conditions and a verification gate into the core loop at its safe points. It does not change how a plain per-turn turn runs.

9. Commands

CLI (src/main.rs)

Flag / command	Effect
--model <MODEL>	Override provider/model for the loop.
--agent <AGENT>	Agent preset driving the loop (default general).
--yes (alias --no-prompt)	Auto-approve all permission prompts.
--no-tui	Headless stdout instead of the TUI.

Flag / command	Effect
<code>--maxsteps <N></code>	Cap agentic loop steps (CLI help text says 500, but with no agent override the effective ceiling is the agent preset's <code>max_steps</code> — default 1024).
<code>--no-git-context / --no-readme-context</code>	Disable git/README context injection into the system prompt.
<code>--dry-run</code>	Readiness check only; never invokes model or tool.
<code>ragent run <PROMPT></code>	One-shot agent execution of a single prompt.
<code>ragent serve [--addr <ADDR>]</code>	Start the HTTP server (default 127.0.0.1:3000).
<code>ragent session resume <ID></code>	Resume a session into a live loop.

TUI slash commands

Command	Effect on the loop
<code>/agent [<name>]</code>	Switch the agent preset driving the loop.
<code>/model [<id>]</code>	Switch model mid-session.
<code>/thinking</code>	Configure thinking mode.
<code>/tools</code>	Tool registry / visibility control.
<code>/compact</code>	Force context compaction of loop history.
<code>/cost</code>	Show the run-cost summary.
<code>/cancel</code>	Cancel in-flight processing.
<code>/clear</code>	Clear the session transcript.
<code>/system</code>	Show the current system prompt.
<code>/undo</code>	Roll back to the last file snapshot.
<code>/yolo</code>	Toggle YOLO trust mode (auto-approve).
<code>/autopilot on [--max-tokens N]</code>	Autonomous loop continuation with token/time budget.
<code>[--max-time N] / off / status</code>	
<code>/reload</code>	Hot-reload subsystems into the running loop.
<code>[agents\ config\ mcp\ skills\ all]</code>	
<code>/context refresh</code>	Clear the prompt-context cache (file tree/git/README recomputed next turn).

HTTP API (Bearer token required; GET /health is open)

Endpoint	Shape
POST /sessions/{id}/messages	Body {"content": String} — the primary loop-driving endpoint.
POST /sessions/{id}/abort	Aborts the running loop.
POST /sessions/{id}/permission/{req_id}	Body {"decision": "allow" "always" "deny"}.
GET /events	SSE event stream (MessageStart, ModelResponse, ToolCallEnd, ...).
POST /sessions/{id}/tasks	Spawn a sub-agent (agent, task, background?, model?).

Keyboard shortcuts

- **Esc** during processing — request a loop interrupt (“stopping after the current stage”).
 - **Ctrl+C** — copy-to-clipboard when text selection is active, else quit arm; **Ctrl+D** confirms quit.
-

10. Worked examples

One-shot CLI run

```
export ANTHROPIC_API_KEY="sk-..."
ragent run --agent coder "Refactor src/main.rs to split the CLI wiring"
```

Driving the loop over HTTP

```
# create a session
SID=$(curl -s -X POST http://127.0.0.1:3000/sessions \
  -H "Authorization: Bearer $RAGENT_TOKEN" \
  -H "Content-Type: application/json" \
  -d '{"directory": "."}' | jq -r .id)

# drive the loop
curl -s -X POST http://localhost:3000/sessions/$SID/messages \
  -H "Authorization: Bearer $RAGENT_TOKEN" \
  -H "Content-Type: application/json" \
  -d '{"content": "List the files in crates/ and summarise each crate"}'

# watch events
curl -N http://localhost:3000/events -H "Authorization: Bearer $RAGENT_TOKEN"
```

Autopilot continuation

```
/autopilot on --max-tokens 50000 --max-time 600
```

The loop continues autonomously until the task completes, the budgets are exhausted, or /autopilot off is issued.

11. Observability

- **Run cost** — every turn publishes `Event::RunCostSummary` with accumulated input/output tokens and cost derived from merged prices entries (`PriceEntry { model, input_per_1m, output_per_1m }`, USD per 1M tokens).
 - **Activity log** — append-only JSONL event store keyed by `RunId`; every user/assistant message and termination reason is recorded.
 - **LLM request stats** — `llmstats slash` command shows request statistics (counts, durations, token tallies).
 - **OpenTelemetry** — `telemetry` section (or legacy `experimental.openTelemetry`) enables OTLP export of session spans.
-

12. Related documentation

- `TUI-QUICKSTART.md` — using the terminal UI (section 5 covers `/loop`)
- `QUICKSTART.md` — installation and first-run configuration
- `SPEC.md` — full configuration schema
- `docs/howtos/loopprogramming.md` — the goal-driven `/loop` extension that wraps this loop with a spec and stop conditions
- `docs/howtos/office.md` — the office + PDF tool families this loop can execute (`tool_visibility.office` gated)
- `docs/howtos/teams.md` — multi-agent team coordination
- `docs/performance/benchmark-guide.md` — benchmarking the loop surfaces